From measuring imbalance to fixing it

You can now **score** a classifier on imbalanced data (L12: PR-AUC, recall, MCC) and **trust** its probabilities (L14: calibration). But the cheese detector still *misses most of the 3% bad batches*.

Can we do better by changing the **data** or the **training** itself — not just the metric we report? This deck: the honest answer.

Two families of fixes: **tilt the training** (class weights / costs) and **reshape the data** (over/under-sampling, SMOTE). Both help *sometimes* — and both have traps.

Outline

What you already have

Tilt the training: weights & costs

Reshape the data: resampling

Does it actually help?

Start with what you already have

Before reshaping data, remember: two tools from L12–L14 handle most imbalance already.

Why imbalance hurts — and your two existing tools

Why it hurts: training minimizes *average* loss, which is dominated by the majority — so the model can score well by mostly ignoring the rare class (L12: 97% accuracy, 0 recall).

You already have two fixes (L12–L14):

- the **right metric** — PR-AUC, recall, MCC, balanced accuracy;
- the **cost-tuned threshold** — move c off 0.5 toward the minority.

Reach for data tricks only after these. Often the metric + threshold are enough — they cost nothing and keep your probabilities honest.

A map of the approaches

Family	**Idea**	**Where**
Algorithm-level	reweight errors / cost matrix in training	**this deck**
Data-level	resample to rebalance the classes	**this deck** (in depth)
Cost-sensitive *thresholding*	move the cutoff by cost	**done in L12/L14**
Ensemble	imbalance-aware bagging/boosting	**ch. 4 (trees)**

This deck covers the first two; the threshold is already yours; imbalance-specific ensembles (BalancedRF, RUSBoost, EasyEnsemble) come with trees in chapter 4.

Tilt the training: weights & costs

Tell the learner that minority mistakes hurt more — without touching the data.

Class weights in the loss

Weight each example's loss by its class, so minority errors count more:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^{n} w_{y^{(i)}} \, L\big(y^{(i)}, f(\mathbf{x}^{(i)})\big), \qquad w_k \propto \frac{n}{K \, n_k}.$$

where n = total examples, K = number of classes, n_k = examples in class k (so $w_{y^{(i)}}$ is the weight of example i's class).

- `class_weight="balanced"` sets exactly $w_k = n/(K\, n_k)$ — the rare class is up-weighted automatically. *(At 3% positive with K=2: minority weight $\approx$ 17, majority $\approx$ 0.5.)*
- It is just **weighted ERM** — works for any loss-based learner (callback Ch. 1).

Same machinery as logistic regression (L11), only the per-example weights change.

The catch: it's roughly a threshold move — and it decalibrates

- For a *well-specified* model, reweighting is **approximately** a shift of the prior / decision threshold (Elkan, 2001) — not a brand-new model.
- With **regularization** the fitted coefficients genuinely change, so it is not an *exact* threshold move — but close.
- Crucially, it **decalibrates**: the model trains as if the minority were common, so its probabilities **inflate** (callback L14).

On a rare-class fit, `class_weight="balanced"` can inflate ECE from near-zero to 0.3 or worse. For a probabilistic model it mostly *duplicates threshold tuning at the cost of calibration* — if you use it and report probabilities, **recalibrate**.

It *does* change the fit meaningfully for tree split / SVM margin criteria — more on "when it helps" later.

Cost-sensitive learning (1/2): costs in the training loss

Class weights use **one weight per class**. The general version puts a price on *each kind* of mistake — a **cost matrix** $C(j, k)$ = cost of predicting j when the truth is k — and folds it into the **loss we minimize**:

$$\hat{\boldsymbol{\theta}} \;=\; \arg\min_{\boldsymbol{\theta}} \; \frac{1}{n} \sum_{i=1}^{n} w_{y^{(i)}} \, L\big(y^{(i)}, f(\mathbf{x}^{(i)})\big), \qquad w_{y^{(i)}} = \text{cost of misclassifying class } y^{(i)}.$$

		true	
		bad (y=1)	good (y=0)
pred.	bad	0	C_{FP}
	good	C_{FN}	0

- **What we minimize:** average *cost-weighted* loss — the same weighted ERM as the class-weights slide, weights now read off the matrix.
- **Class weights are the special case**: cost depends only on the true class ($w_1 = C_{FN}$, $w_0 = C_{FP}$).
- It **changes the fit** (coefficients, tree splits), not just where you cut.

General matrices (cost depends on *which* wrong class) need more — MetaCost, instance-specific costs — beyond this course.

Cost-sensitive learning (2/2): costs at the decision

Costs can also enter at **prediction time** instead of training — keep the fitted model, but turn its probabilities into the **cheapest label**:

$$\hat{y}(\mathbf{x}) \;=\; \arg\min_{j} \sum_{k} P(y{=}k \mid \mathbf{x})\, C(j,k).$$

What is $\hat{y}(\mathbf{x})$? The **label we output** for $\mathbf{x}$ (bad or good) — not a probability. For each candidate prediction j, average its cost over what the truth might be (weighted by the model's probabilities $P(y{=}k \mid \mathbf{x})$), then **pick the cheapest** j.

- For two classes this is **exactly the cost threshold** $c^* = C_{FP}/(C_{FP} + C_{FN})$ from L13 — costs move the cutoff, the **model is unchanged** (needs calibrated $\hat{p}$).

Two knobs, one goal (minimize expected cost): bake costs into the **training loss** (model changes; class weights are the everyday version) *or* apply them at the **decision** (a threshold; model unchanged).

Reshape the data: resampling

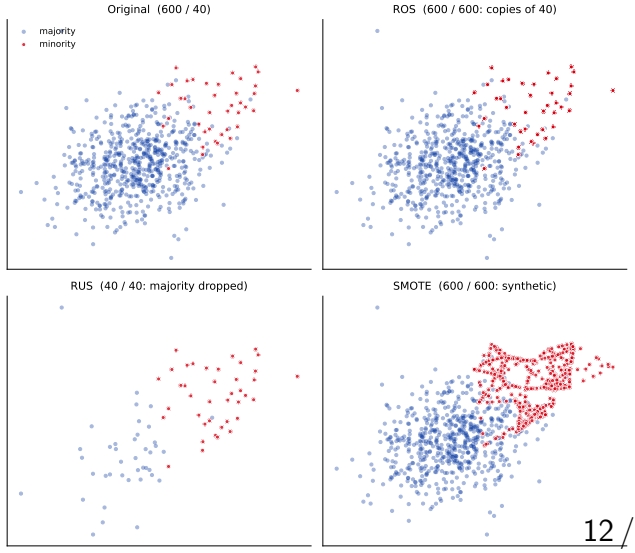
Rebalance the training set itself — classifier-independent, flexible, and the most widely used family. We go deep here; keep the "threshold first" ordering in mind.

Under-, over-, and hybrid sampling

Three ways to rebalance **the training data**:

- **Undersampling** — drop majority examples.
- **Oversampling** — add/synthesize minority examples.
- **Hybrid** — both.

Independent of the classifier ⇒ flexible. See what each does to one blob:

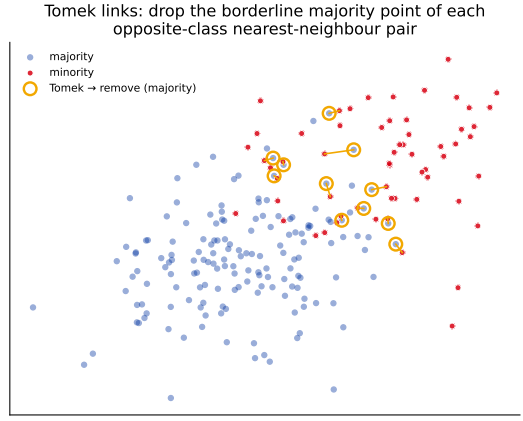


Undersampling: random, then smarter

Random undersampling (RUS) — drop majority points at random. Fast, but may **throw away informative examples**.

Tomek links (smarter) — a pair of *opposite-class nearest neighbours*. They sit on the boundary, so drop the **majority** one to clean it up.

Other cleaning rules exist (edited-NN / NCL, CNN, one-sided selection) — same spirit: remove majority points near the border.



Oversampling: random replication (ROS)

Random oversampling (ROS) — duplicate minority examples until balanced.

- Cheap, classifier-independent, loses no data.
- **But: exact copies** ⇒ the model can **overfit** those few points.

"Doesn't duplicating rows create linear dependence and break the fit?" No — singularity comes from dependent *columns* (features), not repeated *rows*; a copied row just acts as an integer *weight* on that point (`class_weight`, but cruder). What it *does* break is the **i.i.d. assumption** — the copies aren't new information, so the model over-trusts them, **decalibrating** (and leaking if you copy before the split).

Can we add minority examples that are *plausible but new*, instead of copies? That is the idea behind **SMOTE**.

SMOTE: synthesize, don't copy

SMOTE (Synthetic Minority Over-sampling) makes *new* minority points by **interpolating between neighbours**. For a minority point $\mathbf{x}^{(i)}$:

1. find its k nearest *minority* neighbours (default $k = 5$);
2. pick **one of those k at random** — *not* necessarily the closest — call it $\mathbf{x}^{(j)}$;
3. step a random fraction $\lambda \in [0, 1]$ of the way toward it.

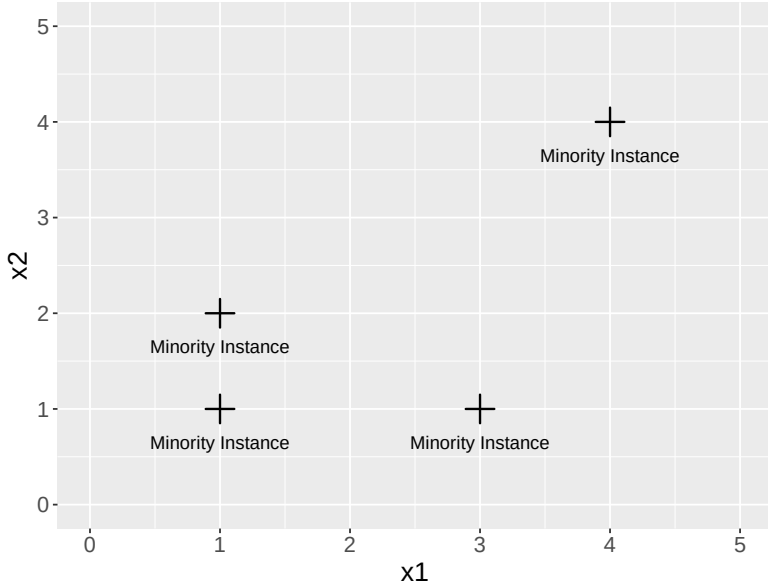
$$\mathbf{x}_{\text{new}} = \mathbf{x}^{(i)} + \lambda \left(\mathbf{x}^{(j)} - \mathbf{x}^{(i)} \right).$$

By hand: $\mathbf{x}^{(i)} = (1, 2)$, the drawn neighbour $\mathbf{x}^{(j)} = (3, 1)$, $\lambda = 0.25$:

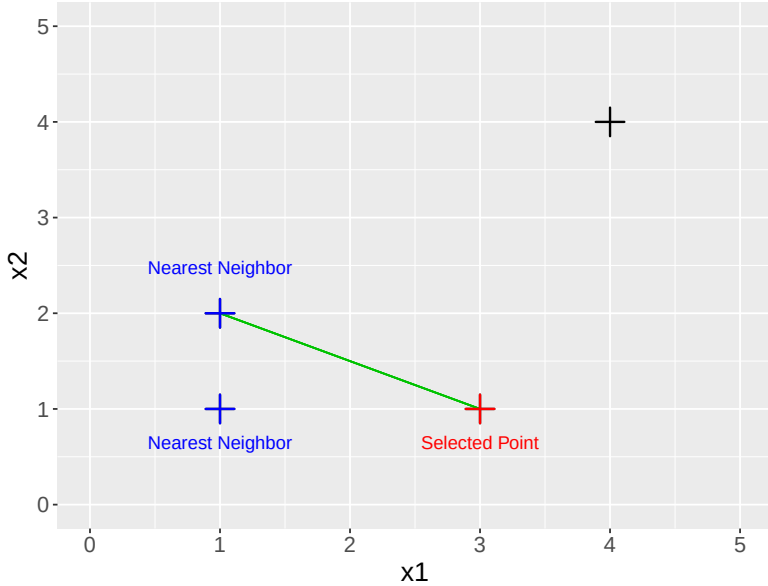
$$\mathbf{x}_{\text{new}} = (1, 2) + 0.25\,(2, -1) = (\mathbf{1.5},\ \mathbf{1.75}).$$

The random neighbour *and* random λ are what make each point new. Next slide: step through several.

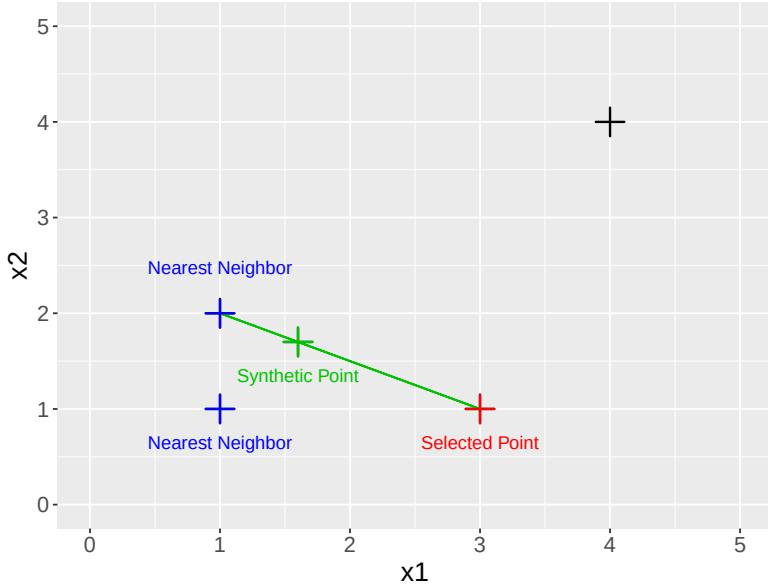
SMOTE: building the points (step through)



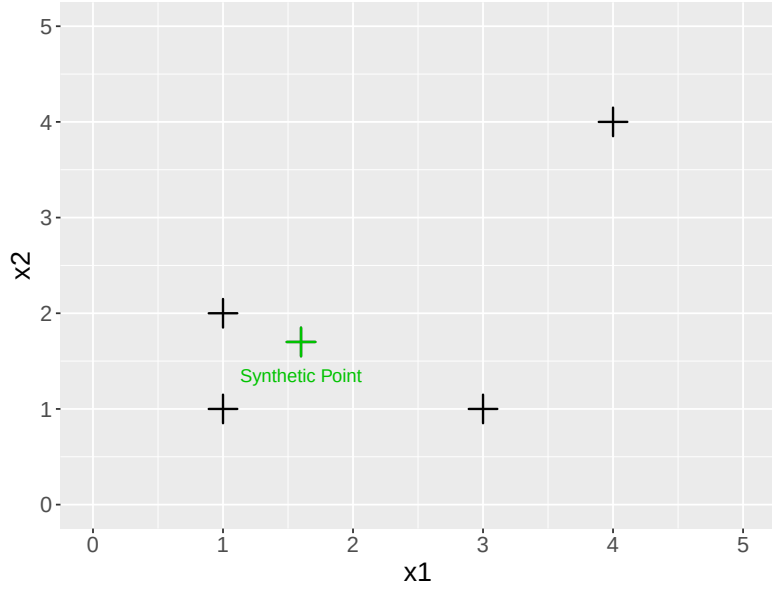
SMOTE: building the points (step through)



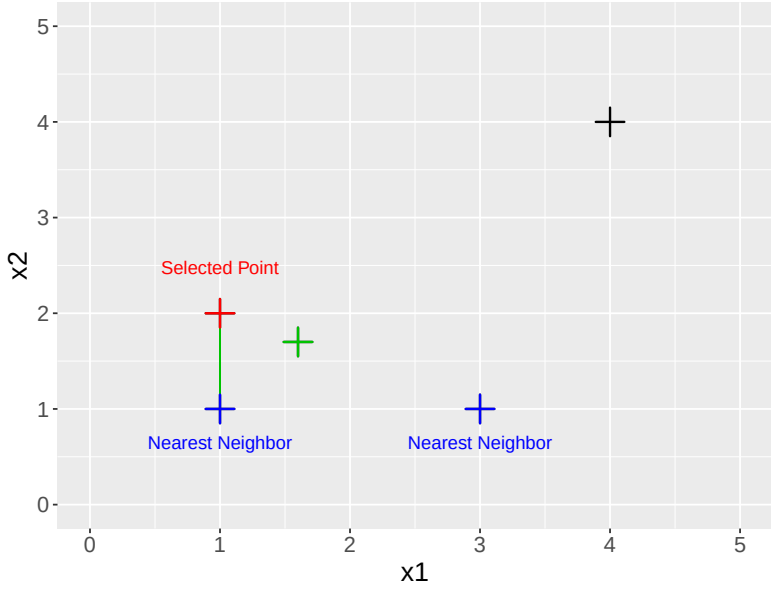
SMOTE: building the points (step through)



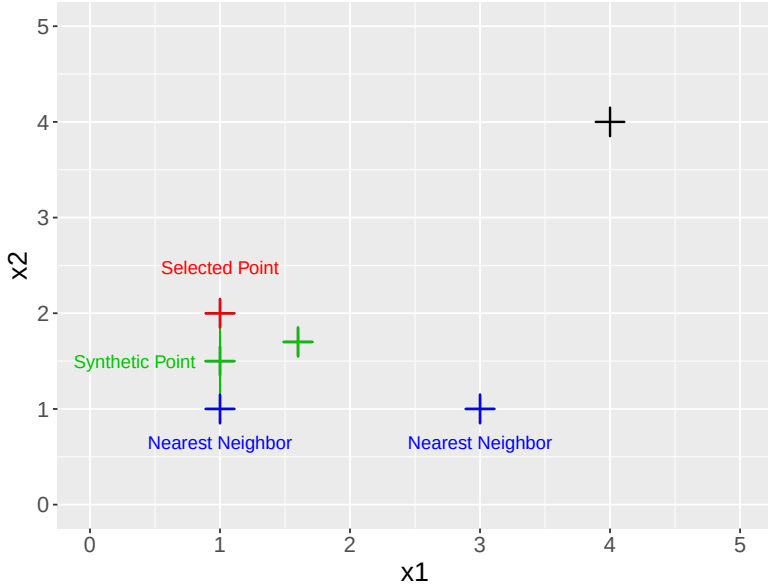
SMOTE: building the points (step through)



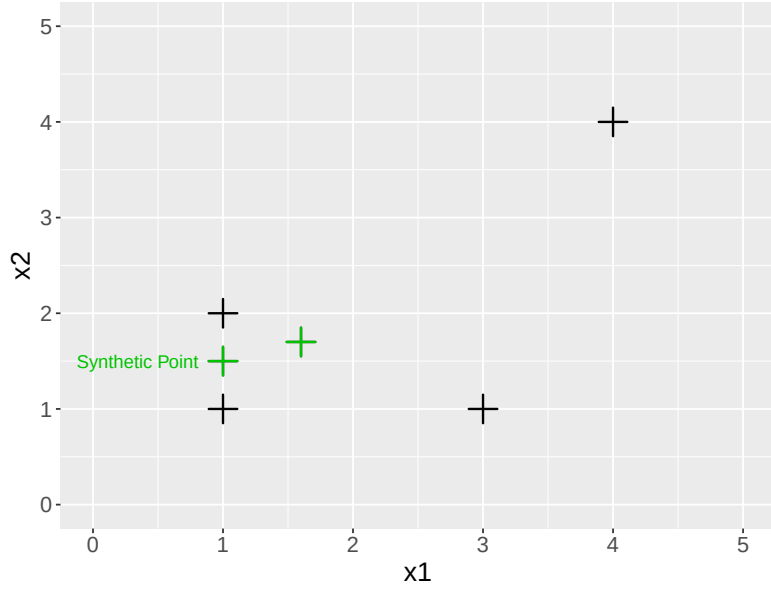
SMOTE: building the points (step through)



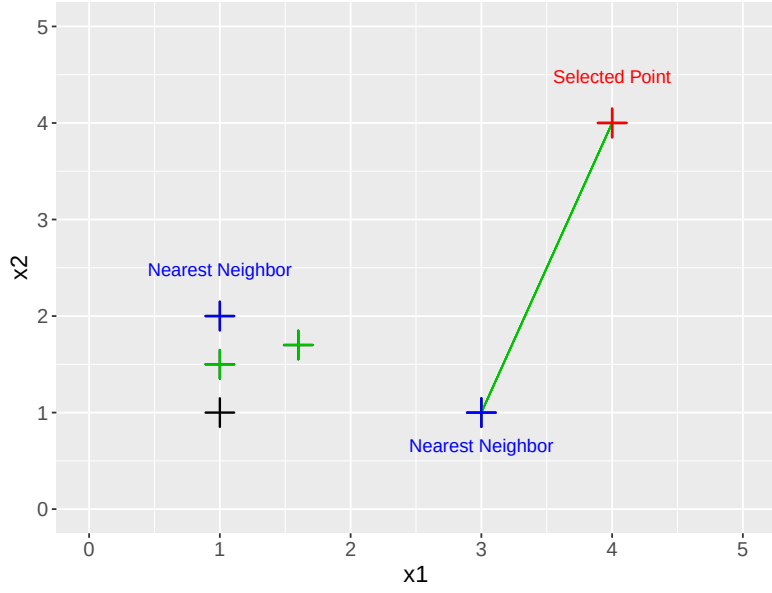
SMOTE: building the points (step through)



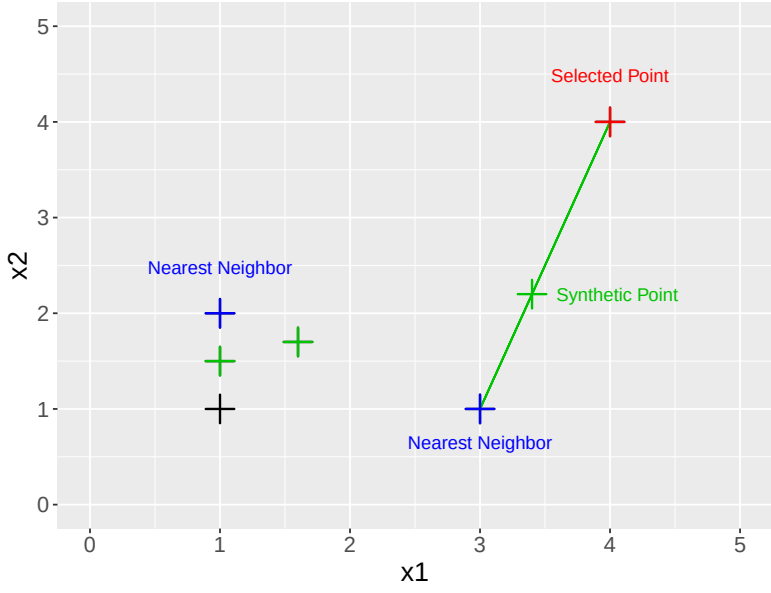
SMOTE: building the points (step through)



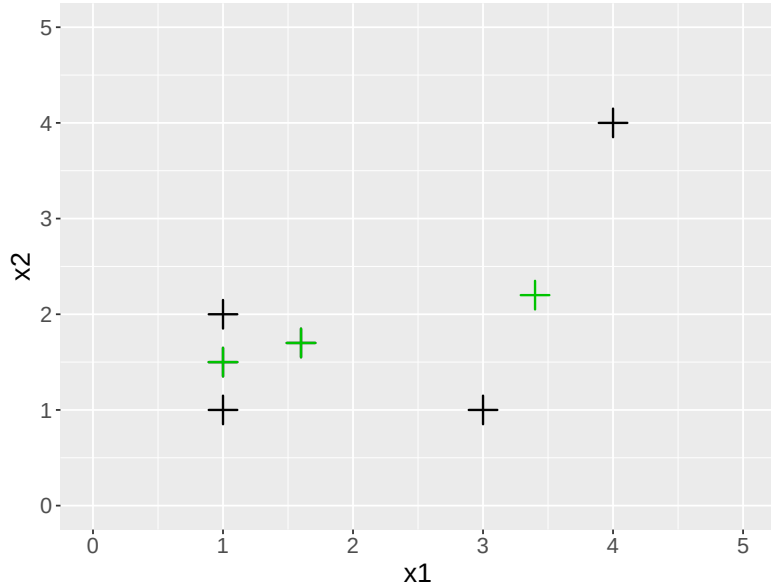
SMOTE: building the points (step through)



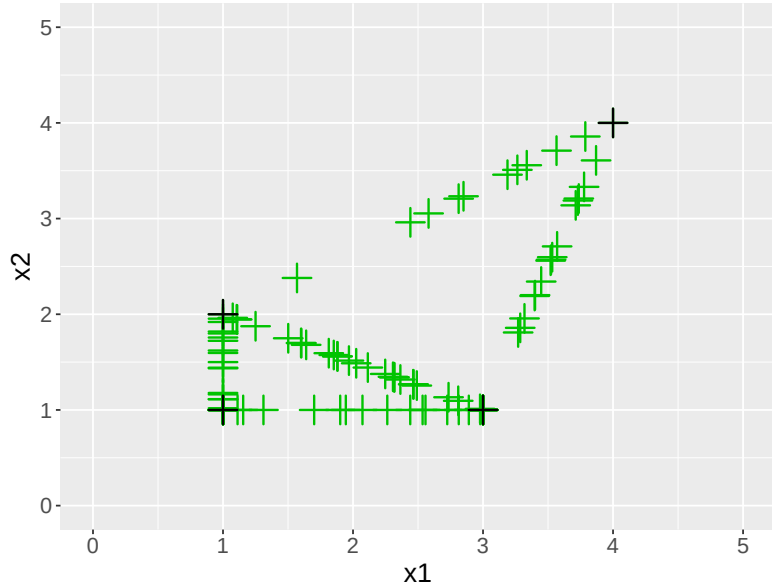
SMOTE: building the points (step through)



SMOTE: building the points (step through)



SMOTE: building the points (step through)



SMOTE in practice

Pros

- Generalizes the minority *region* instead of memorizing a few points (vs ROS).
- The basis for 90+ variants (Borderline-SMOTE, ADASYN, . . .).

Cons

- Ignores the majority ⇒ synthetic points can **bleed across the boundary** into majority territory.
- **Numeric-only** (it interpolates).

Hybrid: pair SMOTE with a cleaning step — **SMOTETomek** (SMOTE then Tomek) or **SMOTEENN** (SMOTE then edited-NN): oversample, then tidy the border.

Categoricals? Plain SMOTE can't interpolate "job = student". Use **SMOTENC** for mixed numeric/categorical data — e.g. the bank-marketing project, with its 9 categorical columns.

Resampling in code — on the training fold only

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline          # NOTE: imblearn's
    Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("smote", SMOTE(random_state=509)),          # resamples TRAIN
        folds only
    ("clf",   LogisticRegression(max_iter=2000)),
])
cross_val_score(pipe, X, y, scoring="average_precision", cv=5)
# class weights instead of resampling:
LogisticRegression(class_weight="balanced")
```

Use `imblearn`'s `Pipeline` so the sampler runs **inside each CV fold** — never resample before splitting (next section: why).

Does it actually help?

When resampling genuinely beats a tuned threshold — and the two traps that bite if you're careless.

When it genuinely beats thresholding

Reweighting / resampling helps when imbalance corrupts the **fit itself**, not just the operating point:

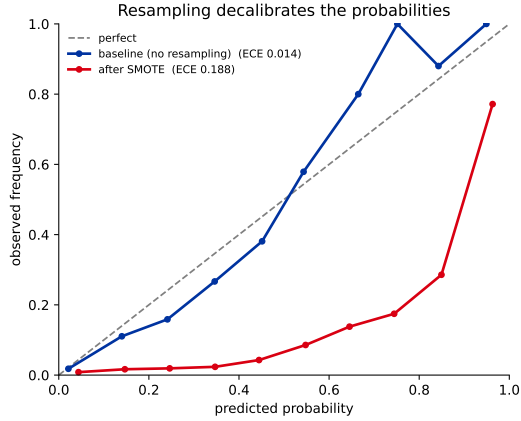
- the minority region is **too sparse for the model to carve a boundary** there;
- **regularization** shrinks the weak minority signal away;
- a **tree's split criterion** picks different splits once the minority is up-weighted.

Rule of thumb: if moving the threshold fixes it, you didn't need the data trick. If the model never *learned* the minority region, resampling/weights might actually help.

Two traps: leakage and decalibration

1. Leakage. Resample **only the training fold**, never before the split — otherwise synthetic/duplicated points leak across train/test and the score is fantasy. Use `imblearn.Pipeline` (previous slide).

2. Decalibration. Resampling changes the base rate the model sees ⇒ its probabilities inflate. **Recalibrate** on untouched data (L14), or report rankings only.



Decision guide

In order:

1. **Right metric + cost-tuned threshold** (L12/L14) — almost always start here.
2. `class_weight` if your learner uses it — then **recalibrate**.
3. **Resampling / SMOTE** only if the minority is truly tiny **and the fit (not the threshold) is the bottleneck** — *inside* CV, then recalibrate.
4. **Get more minority data** / reframe the problem.
5. **Tabular?** reach for trees & boosting (ch. 4), which handle imbalance well.

Resampling is a tool, not a cure — powerful when the fit is starved, wasteful when the threshold would have done the job.

Recap

- Imbalance hurts because average loss is dominated by the majority — but you already fix most of it with the **right metric** + a **cost-tuned threshold** (L12/L14).
- **Class weights / cost-sensitive learning** tilt the training; for a probabilistic model they mostly mimic a threshold move and **decalibrate** ⇒ recalibrate.
- **Resampling**: RUS / Tomek (undersample), ROS (copies → overfit), **SMOTE** (synthetic interpolation; SMOTENC for categoricals).
- Two traps: **resample inside the fold** (leakage) and **recalibrate** (decalibration). Real gains are usually **small**.

Next: decision trees & ensembles (ch. 4) — models that handle non-linearities and imbalance more natively, and bring imbalance-aware ensembles (BalancedRF, RUSBoost).